

RetainIQ

Backend — Poora Code Explanation

Kese code likha, kyu likha, aur sab kese connect hota hai

Stack: FastAPI + SQLAlchemy + Pydantic + SQLite/Postgres + JWT + scikit-learn (+optional XGBoost)
13 routers, 16 DB tables, 17 ML features | Run: uvicorn backend.main:app --reload

Ye document backend ki har file, har API, har formula samjhata hai — simple Hinglish me.

1. Backend kya karta hai?

Frontend sirf dikhata hai; saara dimag backend me hai: login/token, database (customers, transactions, predictions), paise ke formulas (CLV, revenue-at-risk, ROI), ML model (churn probability), explainability (kyu risk hai), recommendations (kaun si action), campaign A/B simulation, CSV import, aur AI analyst chat. Frontend ke har api() call ka jawab yahi banata hai.

Design ke 4 usool:

- Thin routes, fat services: routes/*.py me sirf auth check + validation + function call; asli logic services.py me pure functions ki tarah — test karna aasan.
- Har query tenant-filtered: har table me tenant_id; Demo Bank ka user Demo Fintech ka data kabhi nahi dekh sakta.
- Kabhi down nahi: ML model file na mile to rule-based fallback (_rule) jawab dega; LLM key na ho to deterministic analyst_fallback chalega.
- Har important kaam ka audit: register/login/switch/simulate/campaign/retrain/upload/ai_ask sab AuditLog me likha jata hai.

Tech stack kyu:

Technology	Version	Kyu
FastAPI	0.115.6	Auto docs (/docs), Pydantic validation, async-ready.
SQLAlchemy	2.0.36	ORM: Python class = DB table; SQLite se Postgres tak same code.
Pydantic	2.10.4	Request body ka type-check (schemas.py).
PyJWT	2.10.1	Login token (HS256, 24h expiry). Passwords PBKDF2-SHA256 100k rounds + salt.
pandas/numpy/scikit-learn	2.2.3/1.26.4/1.5.2	Training, scaling, LogisticRegression baseline + HistGradientBoosting (XGBoost mile to wahi).
httpx	0.28.1	Gemini/OpenAI ko server-side call ke liye.
pytest	8.3.4	4 test files — finance, CSV, analyst, full API smoke.
python-multipart	0.0.20	CSV file upload (FormData) ke liye zaroori.
uvicorn	0.34.0	Server jo app chalata hai.

2. File tree

Figure 1 — backend file tree

```

backend/
+-- main.py # app entrypoint, CORS, 13 routers, /api/health
+-- requirements.txt # 11 dependencies
+-- database.py # engine, SessionLocal, Base, get_db()
+-- models.py # 16 SQLAlchemy tables
+-- schemas.py # 6 Pydantic request models
+-- deps.py # current_user, tenant_id, require_role
+-- services.py # ~520 lines: saara domain logic (auth, finance, explain, AI)
+-- seed.py # demo data generator (SEED=42, 600 customers, campaigns)
+-- csv_import.py # banking/telco/generic CSV adapter
+-- ml/train.py # training + metrics.json
+-- ml/infer.py # predict_proba + _rule fallback
+-- ml/explain.py # SHAP stub (asli explain services me)
+-- ml/artifacts/ # model.pkl, scaler.pkl, baseline.pkl, features.json, metrics.json
+-- routes/ auth, dashboard, customers, predictions, segments, recommendations,
    | roi, campaigns, experiments, analytics, ai, data, audit (13 files)
+-- tests/ test_api, test_services, test_csv_import, test_analyst

```

3. Core — main, database, models, schemas, deps

3.1 main.py — entrypoint (26 lines)

Kese likha: import par Base.metadata.create_all(engine) — tables pehli baar auto-ban jati hain, migration ki zaroorat nahi (demo-friendly). FastAPI app banao, CORS lagao (origins env se, default localhost:3000, credentials True taaki browser token wale request bheje), 13 routers include karo, aur 2 utility routes (/api/health, /). Kyu aise: saare route ek file me nahi — har domain ka apna router, main.py sirf jodne ka kaam karta hai.

3.2 database.py — DB connection (17 lines)

DB_URL env DATABASE_URL se, default sqlite:///./retainiq.db. SQLite ko check_same_thread=False chahiye (FastAPI multi-thread), Postgres ko kuch nahi. SessionLocal factory (autoflush/autocommit False — control apne haath me) aur get_db() generator jo har request ko fresh session dekar aakhir me band karta hai (Depends). Kyu generator: request khatam hote hi connection wapas pool me — leak nahi.

3.3 models.py — 16 tables

Har tenant-owned table me tenant_id + index hai. Mukhya tables:

Table	Kya rakhta hai
tenants	id, unique name (Demo Bank, Demo Fintech)
users	tenant_id, email, pw_hash(300), salt(100), role (admin/manager/analyst/viewer); unique(tenant,email)
customers	tenant_id, name, age(35), income(600k), tenure_months(24), region, segment
accounts	tenant_id, customer_id, type, balance
products, customer_products	6 products (Savings, Salary Current, Credit Card, Personal Loan, Insurance, Wealth Invest) + mapping
transactions	tenant_id, customer_id, amount, ts(indexed), type
interactions	kind (complaint/call/login/visit/fail), severity, resolved, resolution_days, note
customer_features	customer_id unique, f JSON (17 features), label (churn 1/0) — ML ka input
churn_predictions	proba(indexed), band, trajectory {d90,d60,d30,today}, drivers JSON, explanation
customer_values	clv, annual_contrib, revenue_at_risk(indexed)
retention_recommendations	action, cost, success, expected_roi, reason, priority, priority_score

Table	Kya rakhta hai
campaigns, campaign_targets	name, intervention, audience JSON, offer_cost, expected_success, status; per-customer group (treatment/control), reached, retained
experiments, experiment_results	campaign ka A/B test + treat_n/ctrl_n/rates/lift/revenue/cost/roi
audit_logs	tenant_id, user_id, action(80), meta(2000), ts(indexed)

Kyu JSON columns (f, trajectory, drivers, audience): features/trajectory ka shape badal sakta hai — har naye feature par migration nahi karni pade. Indexed columns (tenant_id, proba, rar, ts) wahi hain jin par filter/sort hota hai — query fast.

3.4 schemas.py — 6 Pydantic models

RegisterIn (email/password/tenant/role), LoginIn, RoIn (min_proba 0.4, min_clv, segment, intervention_cost 800, success_rate 0.25, reach 1.0), CampaignIn (name, intervention, filters, offer_cost, expected_success, duration), AskIn (question 2–500 chars), UploadRow (reference). Kyu defaults: frontend sliders khaali bheje to bhi API samajhdar jawab de.

3.5 deps.py — shared auth (27 lines)

sec = HTTPBearer(auto_error=False). current_user: header nahi -> 401; decode_token fail -> 401 Invalid; user nahi mila -> 401. tenant_id: user se tenant nikalta hai — har route isi par filter lagata hai (multi-tenancy ek line me). require_role(*roles): admin har jagah ja sakta hai (bypass), baaki ko role match chahiye warna 403. Kyu dependency injection: har endpoint me 10-line auth copy karne ke bajaye Depends(current_user) ek shabd me suraksha.

4. services.py — saara dimag (~520 lines)

Ye file pure functions ka khazana hai — routes isko bulate hain, tests isko check karte hain. Hissso me samjho:

4.1 Auth: hash_pw (PBKDF2-SHA256, 100k rounds, hex), make_token (JWT HS256, 24h expiry, payload {sub, tenant_id, role}), decode_token, audit() jo AuditLog likhta hai (meta 2000 chars tak). Secret env JWT_SECRET se, default dev-secret (production me badlo!).

4.2 Paise ke formulas (tested):

Listing 1 — finance formulas ka sar

```
calc_clv(avg_bal, txn_vol, products, complaints, tenure):
    contrib = avg_bal*3.5% + vol*0.2% + products*1200 - complaints*500 (min 500)
    years = min(7, 3 + tenure/24)
    clv = contrib * years
    calc_rar(proba, clv) = proba * clv
    calc_roi(targeted, success, avg_value, unit_cost, reach=1.0):
        t = round(targeted*reach); retained = t*success; cost = t*unit_cost
        revenue = retained*avg_value; net = revenue-cost
        roi = net/cost (cost 0 ho to 0); break_even = unit_cost/avg_value
    scenarios(base, success): conservative 0.6x, expected 1.0x, optimistic 1.4x (success cap 0.9)
    band(p): >=0.60 High, >=0.35 Medium, else Low
    priority_of(...): score = (0.5*rar/rar_max + 0.3*proba + 0.2*clv/clv_max)*100
    Critical (proba>=0.6, score>=45) | High Priority (proba>=0.4, score>=30)
    | Monitor (proba>=0.35 ya score>=20) | else Low Priority
```

Kyu aise: CLV me balance ka interest-margin + txn fee + product fee minus complaint cost — bank ka asli unit-economics. RaR = umeed ke hisab se kitna paisa khatre me. ROI me break-even rate batata hai 'kitni success

chahiye taaki nuksan na ho'. Scenarios manager ko range dete hain, ek number ka jhootha bharosa nahi.

4.3 pick_action — sabse faydemand ilaaj chunna

8 actions ki table hai — fee_waiver (500, 22%), cashback (1200, 28%), personalized_offer (800, 25%), loan_offer (2000, 20%), premium_upgrade (1500, 18%), rm_call (1000, 30%), service_recovery (700, 35%), no_intervention (0, 2%). Har customer ke liye expected-net (success×CLV – cost) maximize hota hai, plus rule-boosts: complaints>0 -> service_recovery +0.1; balance gir raha -> cashback +0.05; engagement down ya CLV>150k -> rm_call +0.05; single-product -> personalized_offer +0.04. Sath me human-readable reason banta hai ('3 complaints hain isliye service recovery...'). Kyu: har customer ko same offer nahi — jis se sabse zyada expected paisa bache wahi action.

4.4 Features + explainability

17 features: tenure_months, age, income, avg_balance, balance_trend, txn_freq, avg_txn, txn_trend, card_usage, product_count, has_loan, complaints, resolution_days, logins, engagement_decline, inactivity_days, failed_rate. features_from_state() inhe banata hai. explain_fallback(): har feature ka median se deviation nikalo (low-achhe wale: tenure/balance/freq..., high-bure wale: complaints/resolution/eng_decline..., trend wale negative bure), top-3 risk drivers + 2 protective chun kar sentence banao ('txn frequency typical se 40% kam...'). deterioration(): score 0–100 (txn gir raha +25, balance gira +20, card kam +15, engagement down +20, complaints +10 each max 2, failed rate +10). Kyu fallback: SHAP heavy hai — har request par median-comparison millisecond me samajh-dar jawab deta hai.

4.5 audience_query + AI analyst

audience_query(db, tenant, f): Customer JOIN Prediction JOIN Value, tenant filter + min_proba/min_clv/segment — ROI aur campaigns dono isi ko use karte hain (ek hi definition, do jagah). analyst_context(): tenant ka poora summary (counts, RaR, segments, top-20 customers, interventions, campaigns, top-5 products, model metrics). find_customer(): quoted naam ya 'about/for/of + Name' regex se customer pakadta hai. analyst_fallback(): 14-branch router — greeting, specific customer, budget (money parse: lakh/crore/thousand), top-N list, segment, interventions, campaigns, products, trend/why, CLV, model accuracy, counts, definitions (RaR/CLV/ROI/churn), default portfolio summary. Kabhi khaali jawab nahi — hamesha numbers sath. analyst_llm(): GEMINI_API_KEY ho to gemini-1.5-flash, warna OPENAI_API_KEY se gpt-4o-mini (context 6000 chars, 15s timeout); fail ho to fallback. Kyu: LLM nahi bhi ho to app poori chalti hai.

5. ML pipeline — train, infer, explain, artifacts

5.1 ml/train.py (python -m backend.ml.train)

CustomerFeature ke f+label uthao (50 se kam rows ho to skip), 80/20 stratified split (SEED 42), StandardScaler fit, LogisticRegression baseline + primary model (XGBoost mile to XGBClassifier 200 trees, warna HistGradientBoosting). Dono ke AUC/Precision/Recall/F1 print + save: model.pkl, baseline.pkl, scaler.pkl, features.json (17 naam), metrics.json. Current metrics: AUC 0.976, precision 0.85, recall 0.839, F1 0.845, n=7643. Kyu do models: baseline se pata chalta hai complex model kitna behtar hai; scaler zaroori kyuki income (lakh) aur complaints (0–3) ka scale alag hai.

5.2 ml/infer.py — predict_proba

Model+scaler lazy-load (pehli call par), scaled predict_proba 0.01–0.99 clamp. File na mile/corrupt ho to deterministic_rule (sigmoid hand-formula: complaints/eng/inactivity/failed badhaye, tenure/products/logins ghataye) — taaki app kabhi down na ho. Seed/upload/retrain sab isi ek function ko bulate hain.

5.3 ml/explain.py — stub kyu?

shap_drivers() hamesha None return karta hai (SHAP optional import). Asli per-request explanation services.explain_fallback deta hai — fast aur deterministic. SHAP sirf offline analysis ke liye jagah chhodi hai.

6. seed.py aur csv_import.py — data kahan se aata hai

6.1 seed.py (python -m backend.seed [--reset], SEED=42)

Deterministic demo data: 20 first + 15 last naam, 8 Indian cities, 6 products, 4 users (admin/manager × Demo Bank/Demo Fintech, password demo123). make_customers: loyal 45% (lambi tenure, regular txn), declining 25% (girta balance, complaints — 75% churn label), volatile 15% (30% churn), new 15% (5% churn). Segment: income ≥25L HNI, ≥10L Affluent, 12% SME, baaki Mass. Har customer: 1–3 accounts, products, 12 mahine decaying transactions, complaint+engagement interactions. Phir predict_proba + CLV/RaR + Prediction/Value/Recommendation rows; 3 completed campaigns (Festive Win-back, HNI Save Desk, Service Recovery Blitz — 40 targets, 80/20 split, Bernoulli retained) + 1 draft. Bank 400 + Fintech 200 = 600 customers. Kyu deterministic: har baar same data — demo aur tests repeatable.

6.2 csv_import.py — banking/telco/generic adapter

detect_dataset_type(): telco agar {customerid, monthly charges} + churn column; banking agar saare BANKING_COLS; generic agar id/name; warna unknown (400 reject). Telco mapping: tenure clamp, monthly/total defaults, senior->age, income = monthly×12×contract-mult + CLTV/2, txn_freq phone/internet/support se, complaints support-tickets se, segment CLTV se, label Churn Value/Label se; plus behavioural proxies (logins, inactivity, failed_rate, trends, card, engagement). _num '\$1,200.50'/NaN/blank sambhalta hai; _det deterministic fill deta hai; duplicate/missing-id rows skip + issues list. Kyu: user koi bhi CSV laye — banking, Telco-churn sample, ya generic — DB me ek hi normalized shape jaye.

7. API reference — 13 routers, har endpoint

Sab routes me Depends(get_db) + Depends(current_user) hota hai (neechे Auth column). tenant_id se har query filter hoti hai.

7.1 Auth (/api/auth)

Method + Path	Auth	Logic
POST /api/auth/register	public	Tenant dhundo/banao; email duplicate to 400; salt+hash; user banao; audit; token do
POST /api/auth/login	public	Email se user; hash verify, galat to 401; audit; token do
GET /api/auth/me	user	Apna email/role/tenant + admin ho to saare tenants
POST /api/auth/switch?tenant_id=	user	Demo ke liye tenant badlo (bina membership check); naya token

7.2 Dashboard (/api/dashboard) — 7 GET

Endpoint	Jawab
GET /summary	total/high/medium/low, RaR sum, expected_protected, campaign_cost, ROI%, budget 10L const, targeted/retained counts, campaigns
GET /risk-dist	[[band,count]]
GET /revenue-by-segment	[[segment,rar,count]]
GET /product-risk	[[product,customers,high_risk,rar]] sorted -rar
GET /risk-trend	trajectory arrays ka mean: [[point:d90..today, avg_risk, high count]]
GET /quadrant	{median_clv, hv_hr, hv_lr, lv_hr, lv_lr} (CLV median + proba 0.5 split)
GET /campaign-perf	[[id,name,status,roi_pct,revenue,lift]]

7.3 Customers (/api/customers)

Endpoint	Logic
GET /api/customers?search&risk&segment&sort&page&page_size=20	4-table join; ilike search; band/segment filter; python sort (rar/proba/clv/name); paginate -> {total,page,items[...action,priority,score]}
GET /api/customers/{id}	404 agar doosre tenant ka; prediction/value/rec/feature/accounts/12 txn/10 interactions/products; deterioration; full 360 JSON (drivers, explanation, trajectory, action{cost,success,roi}, balances, txns)

7.4 Predictions, Segments, Recommendations, ROI

Endpoint	Logic
GET /api/predictions/metrics	metrics.json hai to {trained, auc, precision, recall, f1, confusion, n}, warna {trained:False + retrain hint}
POST /api/predictions/retrain (admin/manager/analyst)	train.main() phir tenant ki saari predictions/values/recs rewrite (medians->proba->CLV->explain->action); audit; {ok, updated}
GET /api/segments/overview	median CLV + 4 quadrants + per-segment [[segment,count,rar,avg_proba]] sorted -rar
GET /api/recommendations?priority&limit=100	4-table join, priority filter, -score sort, limit slice
POST /api/roi/simulate (Roiln)	audience_query -> avg_clv -> calc_roi + scenarios -> audit -> {audience,avg_clv,estimated,targeted,retained,cost,revenue,net,roi,roi_pct,break_even,scenarios}

7.5 Campaigns + Experiments

Endpoint	Logic
GET /api/campaigns	List desc: [[{id,name,intervention,audience,offer_cost,expected_success,status,targets}]]
POST /api/campaigns (admin/manager, CampaignIn)	audience_query + avg_clv -> draft Campaign -> projected {targeted,revenue,cost,roi_pct}; audit
GET /api/campaigns/{id}	Draft ho to audience note; launched ho to Experiment+Result {treat/ctrl n, rates, lift, revenue, cost, roi}
POST /api/campaigns/{id}/launch (admin/manager)	Deterministic Random(42+cid): shuffle -> 20% control (min 5); Bernoulli retained (control 15% vs expected_success); Result: lift×tn×avgv revenue; status completed; audit
GET /api/experiments, GET /api/experiments/{id}	A/B ledger + detail (read-only; note: simulated outcomes)

7.6 Analytics, AI, Data, Audit

Endpoint	Logic
GET /api/analytics/products	Dashboard product-risk reuse (DRY)
GET /api/analytics/deterioration	score>=30 wale top-50 [{id,name,score,flags,proba}]
GET /api/analytics/alerts	risk_spike (today-d30>=0.15), danger_zone (proba>=0.6 + CLV>=150k), low_roi (roi<0.2); top 30
GET /api/ai/suggested (public)	5 canonical questions
POST /api/ai/ask (AskIn)	context + find_customer -> analyst_llm ya fallback -> audit -> {answer, grounded, disclaimer}
GET /api/data/sample (public)	2-row banking CSV text
GET /api/data/status	{customers, features} counts
POST /api/data/preview (form f)	CSV parse -> normalize -> {type,rows,importable,rejected,churn_labels,mapping,sample,issues,telco_note?}
POST /api/data/upload (admin/manager/analyst)	Dedupe (existing names skip) -> pass1 insert Customer/Feature/Account -> medians -> pass2 Prediction/Value/Rec (priority Monitor, score p*50) -> audit -> {added,rejected,errors,next:retrain hint}
GET /api/audit?limit=50	Tenant ke [{action,meta,ts}] desc

8. Architecture + data flow + chalana + tests

Request ka safar:

Figure 2 — request lifecycle

```

Browser (Bearer JWT)
-> CORS middleware
-> deps: HTTPBearer -> current_user (decode + DB lookup, 401/403) -> [require_role]
-> routes/*.py: Pydantic validation (RoiIn/CampaignIn/AskIn/UploadFile) ya query params
-> services.*: hash/make_token | audience_query | calc_roi/scenarios |
calc_clv/rar/band/priority
| pick_action | features/explain/deterioration | analyst_* | audit
-> models.* via db session: tenant-filtered SELECT/JOIN/INSERT/UPDATE
-> ML: infer.predict_proba (model.pkl+scaler ya _rule) | train.main (retrain par)
-> CSV: csv_import.normalize (banking/telco/generic) | LLM: httpx Gemini/OpenAI (optional)
-> db.commit + AuditLog
-> JSON jawab (+ estimated / grounded / simulated flags + disclaimer: estimates, guarantees
nahi)

```

Do examples: (1) POST /api/roi/simulate: JWT -> RoiIn -> audience_query -> calc_roi+scenarios -> audit -> jawab.
(2) POST /api/data/upload: JWT+role -> CSV -> normalize -> dedupe -> insert -> medians -> har row
predict+CLV+explain+action -> audit -> {added,rejected} -> client retrain bulaye to model labels seekhta hai.

Kaise chalaye:

Listing 2 — backend commands

```

cd backend
pip install -r requirements.txt
python -m backend.seed # 600 demo customers (pehli baar)
uvicorn backend.main:app --reload # http://localhost:8000, docs: /docs
python -m backend.ml.train # dobara training (optional)
pytest backend/tests -q # 4 test files

```

Tests kya check karte hain: test_services (ROI math, zero-cost, CLV/RaR, band/priority Critical, complaints->service_recovery, explain≥3 drivers), test_csv_import (banking unchanged, telco 6->4 rows, deterministic, \$1,200.50 parse, unknown reject), test_analyst (greeting, Rs.5 lakh parse, top-N, customer detail 87%, money 2 crore=20M), test_api (smoke: login->dashboard>100->detail->ROI->campaign create+launch->experiments->AI len>20).

Production notes: JWT_SECRET aur DATABASE_URL (Postgres) env me set karo; CORS_ORIGINS me asli frontend domain; retainiq.db aur .env commit mat karo (.gitignore me hain).